

POISSON

Actividad 1.

La estructura de tus archivos es excelente para un proyecto de simulación computacional. A continuación, detallo el impacto técnico de las directivas que utilizaste para que puedas nutrir las conclusiones de tu informe.

1. Implementación de `#pragma omp parallel for`

¿Qué hace en el código?

Al colocar esta directiva justo antes del bucle exterior `for (int i = 1; i < M; ++i)`, le indicas a OpenMP que divida las iteraciones de ese bucle entre los hilos (threads) disponibles.

En la versión secuencial, la malla se recorre de manera estrictamente lineal (fila por fila, de arriba hacia abajo). En la versión paralela, el dominio se divide en bloques de filas. Si tienes 4 hilos y $M = 50$, el primer hilo procesará aproximadamente las filas 1 a 12, el segundo las filas 13 a 24, y así simultáneamente.

2. La necesidad de la cláusula `reduction(max:delta)`

¿Por qué es necesaria?

La variable `delta` rastrea la diferencia máxima entre el valor antiguo y el nuevo de la temperatura ($T_{i,j}$) para determinar si la simulación ha convergido respecto a tu parámetro de tolerancia (`TOL_param`).

Si omites la reducción, tendrás una **condición de carrera (data race)**. Múltiples hilos intentarían leer y sobrescribir la variable compartida `delta` al mismo tiempo cuando encuentran una diferencia mayor. Esto corrompería el valor de `delta` y la simulación podría detenerse prematuramente o entrar en un bucle infinito.

Al usar `reduction(max:delta)`, OpenMP crea una copia local y privada de `delta` para cada hilo. Al finalizar el bucle paralelo, OpenMP toma automáticamente el valor máximo de todas las copias privadas y lo asigna a la variable global `delta` de forma segura.

3. Impacto en el código y comportamiento numérico

Al analizar el paso del código secuencial al paralelo, ocurren dos fenómenos importantes:

- **Impacto en el rendimiento (Tiempo):** Al distribuir la carga computacional, el tiempo de ejecución por iteración disminuye significativamente, escalando en función del número de

núcleos físicos disponibles (aceleración computacional).

- **Impacto en el número de iteraciones:** Tu código secuencial actualiza la matriz `T` *in-place*. Esto significa que cuando calculas $T_{i,j}$, utilizas el valor $T_{i-1,j}$ que acaba de ser actualizado en el paso anterior. Esto se conoce matemáticamente como el método de **Gauss-Seidel**. Al paralelizar con OpenMP, los hilos calculan diferentes secciones de la malla al mismo tiempo. Un hilo podría estar leyendo una frontera que otro hilo aún no ha actualizado, o viceversa. Esto rompe la dependencia estricta de Gauss-Seidel, transformando el comportamiento del algoritmo en un híbrido entre Gauss-Seidel y Jacobi (Block-Jacobi). **En consecuencia, es altamente probable que el número total de iteraciones cambie** (usualmente aumenta ligeramente) para alcanzar la misma tolerancia de $1e-6$, aunque el tiempo total de ejecución será menor gracias al paralelismo.

Este comportamiento de las iteraciones es un detalle técnico crucial que los docentes suelen evaluar en este tipo de prácticas de sistemas distribuidos.

Actividad 2

Aquí te explico qué está pasando "bajo el capó" y cómo esto impacta tu código en comparación con la versión anterior:

1. ¿Qué hace exactamente `collapse(2)` ?

En la actividad anterior, solo paralelizaste el bucle externo (las filas `i`). Si tenías 4 hilos y $M = 50$, cada hilo recibía un bloque de unas 12 filas. El bucle interno (las columnas `j`) seguía siendo estrictamente serial para cada hilo.

Al usar `collapse(2)`, le ordenas al compilador que **fusion**e los dos bucles anidados en uno solo.

- **Antes:** Tenías $M - 1$ unidades de trabajo para repartir.
- **Ahora:** Tienes $(M - 1) \times (N - 1)$ unidades de trabajo.

Para una malla de 50×50 , pasas de repartir 49 tareas a repartir **2,401 tareas**.

2. Impacto en el Rendimiento

- **Mejor balance de carga (Load Balancing):** Al tener muchas más tareas pequeñas en lugar de pocas tareas grandes, es más probable que todos los hilos terminen al mismo tiempo. Es más difícil que un hilo se quede "ocioso" mientras los demás trabajan.
- **Reducción de Overhead:** OpenMP tiene más flexibilidad para asignar iteraciones a los hilos, especialmente útil si el número de hilos es cercano al número de filas.
- **Costo de cálculo:** Nota que en tu código definiste `const double h2 = h * h;` fuera del bucle. Esto es excelente, porque al colapsar bucles, quieres que el cuerpo del ciclo sea lo más ligero posible.

3. Reflexión para tu informe: ¿Es siempre recomendable?

No siempre. Se recomienda usar `collapse` cuando:

1. El bucle externo tiene **pocas iteraciones** (menor al número de hilos). Por ejemplo, si tienes 16 hilos pero solo 10 filas, sin `collapse` habría 6 hilos sin hacer nada.
2. Los bucles son **perfectamente anidados** (no hay código entre el primer `for` y el segundo).

Posible error/observación: Si la malla es muy pequeña (como 50×50), la diferencia de tiempo podría ser despreciable o incluso un poco más lenta debido al costo de gestionar tantas tareas. La verdadera potencia de `collapse(2)` se ve en mallas grandes (500×500 o superiores).

Detalles Técnicos en tu implementación:

Veo que mantuviste la `reduction(max:delta)`. Esto es **fundamental**, ya que ahora el riesgo de condición de carrera es aún mayor al tener hilos saltando por toda la matriz tanto en filas como en columnas simultáneamente.

Un pequeño detalle de optimización que hiciste muy bien:

C++

```
const double h2 = h * h;
const double k2 = k * k;
const double denom = 2.0 * (h2 + k2);
```

Al pre-calcular el denominador y las potencias fuera del bucle `while`, evitaste miles de multiplicaciones redundantes dentro de la región paralela. ¡Muy bien pensado!

Actividad 3.

Esta es una de las partes más interesantes de la programación paralela: pasar del paralelismo de **datos** (hilos haciendo lo mismo sobre diferentes partes de un arreglo) al paralelismo de **tareas** (hilos haciendo cosas totalmente distintas al mismo tiempo).

Aquí tienes el análisis para tu **Actividad 3**:

1. Implementación de `#pragma omp sections`

En tu código, has envuelto dos funciones bajo una región de secciones:

- **Sección 1:** `initialize_boundaries(M, N, T, h, k);`
- **Sección 2:** `poisson_source(M, N, source);`

¿Qué está pasando realmente?

Cuando el hilo principal llega al bloque `parallel sections`, "recluta" a los hilos disponibles.

- El **Hilo A** se encarga de calcular todas las potencias y llenar los bordes de la matriz `T`.
- El **Hilo B** se encarga de llenar la matriz `source` con el valor constante `4.0`.
- Ambos trabajan **simultáneamente**. El programa no avanzará a `solve_poisson` hasta que ambos hilos hayan terminado su respectiva sección (hay una barrera implícita al final del bloque).

2. Respuesta a las Tareas y Reflexiones

¿Tuviste que reordenar las funciones? ¿Por qué?

En este caso específico, **no es necesario reordenarlas**, pero sí es obligatorio que sean **independientes**.

- `initialize_boundaries` modifica la matriz `T`.
- `poisson_source` modifica la matriz `source`.

Como ninguna función depende de los resultados de la otra para completar su trabajo, pueden ejecutarse en cualquier orden o al mismo tiempo sin riesgo de condiciones de carrera. Si `poisson_source` necesitara datos ya calculados por las fronteras, **no** podrías meterlas en un bloque de secciones.

¿Vale la pena paralelizar funciones tan rápidas?

Siendo sinceros y pragmáticos: **Para una malla de 50×50 , probablemente no.**

- **El costo de gestión:** Crear una región paralela y asignar hilos toma un tiempo (overhead). Si la función tarda microsegundos, el "papeleo" administrativo de OpenMP podría tardar más que la ejecución misma.
- **Escalabilidad:** Esto cobra sentido en simulaciones de **Mecánica de Fluidos** o **Física de Materiales** donde la inicialización implica leer archivos gigantescos de disco o realizar cálculos analíticos complejos para cada celda.

3. Impacto en el Código

- **Estructura:** El código se vuelve más modular. Estás separando claramente la "Preparación" de la "Resolución".
- **Rendimiento Global:** Verás que el tiempo de "setup" se reduce a la mitad (asumiendo que ambas funciones tardan lo mismo), pero como el grueso del tiempo se lo lleva el `while` en `solve_poisson`, el impacto en el tiempo total del programa será pequeño.

Un detalle técnico para tu informe: En tu implementación de `solve_poisson`, mantuviste el `collapse(2)` de la actividad anterior. Eso es excelente, porque significa que estás combinando **paralelismo de tareas** (en el inicio) con **paralelismo de datos** (en el cálculo), lo cual es una práctica profesional de optimización.

Actividad 4 y 5

Entendido, Sebastián. Vamos a centrarnos exclusivamente en el análisis técnico de estas dos actividades. Estas representan los dos pilares del control en OpenMP: **cómo se reparte el trabajo** (Schedule) y **cómo se protegen los datos** (Sincronización).

Aquí tienes el análisis para tu informe:

Actividad 4: Estrategias de Planificación (`schedule`)

En esta implementación, el foco está en la eficiencia del reparto de carga entre los núcleos.

- **Implementación Técnica:** Has separado la región `#pragma omp parallel` del bucle `#pragma omp for`. Esto permite que los hilos se creen una sola vez al inicio de cada iteración del `while`, reduciendo ligeramente el "ruido" de gestión de hilos.
- **Análisis del `schedule(static)` :**

- **Mecánica:** Divide las filas de la matriz ($M - 1$) en bloques de tamaño igual y los asigna a los hilos de forma fija antes de empezar el cálculo.
 - **Impacto en Poisson:** Es la estrategia **óptima** para este problema. Como cada celda de la grilla requiere exactamente la misma cantidad de operaciones matemáticas (sumas y multiplicaciones del esquema de 5 puntos), no hay hilos que terminen mucho antes que otros. El *overhead* o sobrecarga de gestión es mínimo.
 - **Comparativa Teórica:** Si usaras `dynamic`, el programa perdería tiempo en "negociaciones" constantes entre hilos para ver quién toma la siguiente fila, lo cual no tiene sentido aquí porque el trabajo es perfectamente uniforme.
-

Actividad 5: Control de Sincronización (`critical`)

Aquí el análisis cambia de la eficiencia del reparto a la **integridad de los datos**.

- **Implementación de Reducción Manual:** En lugar de usar la cláusula automática `reduction`, has implementado una reducción manual. Cada hilo calcula su propio residuo máximo en una variable privada llamada `local_delta`.
 - **Análisis del `#pragma omp critical(update_delta)`:**
 - **Mecánica:** Esta directiva crea un "embudo" o sección de exclusión mutua. Solo un hilo a la vez puede entrar a comparar su `local_delta` con el `delta` global.
 - **Uso de Nombres:** Es excelente que hayas usado un nombre para la sección crítica: `(update_delta)`. Esto evita que otros procesos críticos (si los hubiera) se bloqueen entre sí innecesariamente.
 - **Impacto en el Rendimiento:**
 - **Punto Crítico:** Al colocar la sección crítica **fuera** de los bucles `for` de la matriz, el impacto en el rendimiento es casi nulo. Si la hubieras puesto dentro del bucle `j`, el programa sería extremadamente lento porque los hilos se detendrían miles de veces por segundo para esperar su turno.
 - **Seguridad:** Garantiza que el valor de `delta` (el criterio de parada) sea siempre el máximo real, evitando que dos hilos sobrescriban el valor al mismo tiempo y produzcan un resultado erróneo.
-

Resumen para tu tabla de resultados:

Versión	Ventaja Técnica	Posible Desventaja
Schedule Static	Mínima sobrecarga. Aprovecha la uniformidad de la grilla de Poisson.	Ineficiente si el trabajo por celda fuera variable (ej. fractales).
Critical Section	Control total sobre la actualización de variables globales complejas.	Si se usa mal (dentro de bucles internos), destruye el paralelismo.

Un detalle para tu informe: En la versión `critical`, el contador `iterations++` está fuera de la región paralela. Esto es correcto porque solo el hilo maestro debe llevar la cuenta de cuántas veces ha convergido la matriz completa.

Actividad 6

En esta **Actividad 6**, entramos en el terreno de la optimización de bajo nivel. Has implementado la directiva `#pragma omp atomic`, que es la "hermana menor" y más veloz de `critical`.

Aquí tienes el análisis de cómo esta implementación afecta el comportamiento de tu solver de Poisson:

1. Implementación de `#pragma omp atomic`

En tu código, has aplicado `atomic` sobre la variable `iterations++`.

¿Qué está pasando físicamente en el procesador?

A diferencia de `critical`, que rodea un bloque de código con un "candado" (mutex), `atomic` utiliza instrucciones directas del hardware del CPU. Es una operación indivisible: el hilo lee el valor, lo aumenta y lo escribe en la memoria en un solo paso técnico, sin permitir que ningún otro hilo se cruce.

2. El combo `single + atomic`

Es muy interesante cómo estructuraste esta parte:

C++

```
#pragma omp single
{
    #pragma omp atomic
```

```
iterations++;  
}
```

- **#pragma omp single**: Como estamos dentro de una región paralela, si no usaras `single`, **cada hilo** intentaría aumentar el contador. Si tienes 8 hilos, ¡el contador subiría de 8 en 8! `single` asegura que solo un hilo (el primero que llegue) ejecute el incremento por cada barrido de la malla.
- **¿Es necesario `atomic` aquí?**: Siendo estrictamente técnicos, si ya estás dentro de un bloque `single`, no habría condición de carrera porque solo un hilo está ejecutando esa parte. Sin embargo, usarlo es una excelente práctica de "programación defensiva" para asegurar que la actualización de la memoria sea atómica y visible para todos los hilos inmediatamente.

3. Diferencias: `atomic` vs `critical`

Para tu informe, esta comparación es clave:

Característica	<code>#pragma omp critical</code>	<code>#pragma omp atomic</code>
Alcance	Puede proteger bloques enteros de código, llamadas a funciones y lógica compleja.	Solo protege una instrucción de actualización de memoria (<code>++</code> , <code>--</code> , <code>+=</code> , etc.).
Rendimiento	Más lento. Tiene mucha sobrecarga porque el sistema operativo debe gestionar el bloqueo.	Mucho más rápido. Se ejecuta casi a la velocidad del hardware puro.
Flexibilidad	Alta. Puedes poner cualquier cosa dentro.	Baja. Solo para operaciones aritméticas básicas.

4. Impacto en el solver de Poisson

Al usar `atomic` para el contador de iteraciones y `reduction` para el error (`delta`), has creado una versión muy eficiente. Al ser una operación tan pequeña (un simple incremento), el impacto en el tiempo total es mínimo comparado con el cálculo de la matriz, pero es la forma correcta y profesional de manejar contadores globales en sistemas distribuidos.

Nota sobre Gauss-Seidel:

En tu comentario del código mencionas "Gauss-Seidel asíncrono". Es un gran término. Al actualizar `T[i][j]` *in-place* dentro de una región paralela, los hilos están usando valores "nuevos" de sus vecinos de forma impredecible. Esto acelera la convergencia en comparación

con el método de Jacobi puro, aunque el número de iteraciones sea ligeramente distinto al de la versión serial.

¿Te parece si cerramos con la **Actividad 7**, donde exploramos el uso de **Tasks** y dividimos la grilla en bloques? Ese es el nivel avanzado de OpenMP.